CROWD DYNAMICS IN THREE GOES · HANDBOOK

Go 2: Build and Implement: crowd models

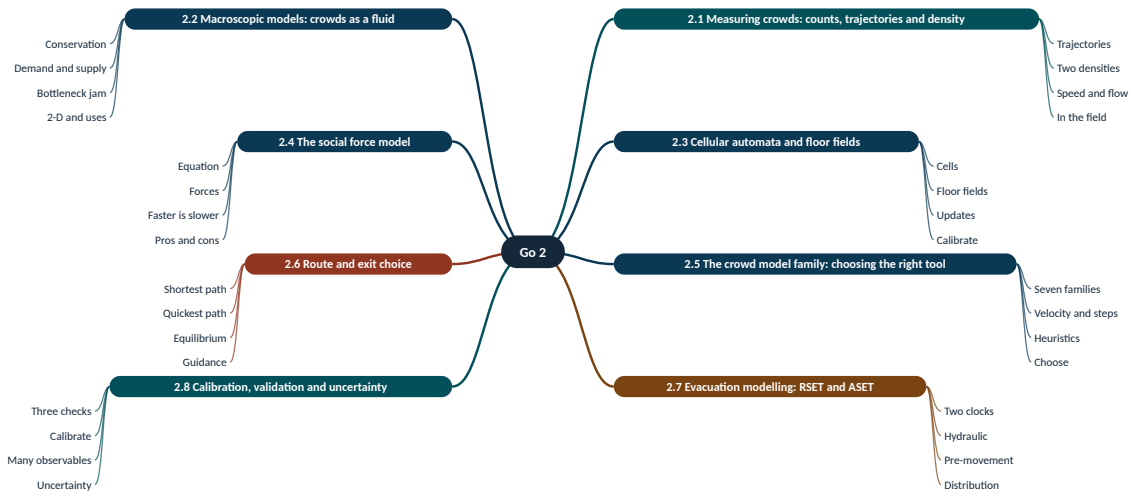
8 modules · about 5.9 hours of study

Companion to the website. Explorers, simulations, challenges and runnable code are online; this handbook keeps the text, worked numbers, mind maps and questions for reading offline.

Contents

1. **2.1 Measuring crowds: counts, trajectories and density** · 45 min
   Every model starts from measurements. Learn to get density, speed and flow from counts and trajectories, and why the Voronoi method gives smoother, fairer densities than counting heads in a box.
2. **2.2 Macroscopic models: crowds as a fluid** · 45 min
   Treat the crowd as a continuous density, conserve people, and let the fundamental diagram decide the flow. A jam forms at a bottleneck and its tail travels backwards, exactly as the numbers predict.
3. **2.3 Cellular automata and floor fields** · 45 min
   Divide the floor into 40 cm cells, one person per cell. Each step, people hop to a neighbouring cell, guided by a static field (distance to the exit) and a dynamic field (others' trails).
4. **2.4 The social force model** · 50 min
   Each person is pulled towards a goal and pushed by neighbours and walls, like particles on springs. The model reproduces lanes, clogging at doors and 'faster is slower', but needs careful parameters.
5. **2.5 The crowd model family: choosing the right tool** · 40 min
   Hydraulic calculations, macroscopic, cellular, force-based, velocity-based, step-based and heuristic models each answer different questions. Choose by the question, the scale and the data you have.
6. **2.6 Route and exit choice** · 40 min
   Where people go matters as much as how they walk. Shortest-path, quickest-path and equilibrium rules give different crowd patterns; good guidance moves the crowd towards the best split.
7. **2.7 Evacuation modelling: RSET and ASET** · 45 min
   An evacuation is safe when the time people need (RSET) is less than the time available (ASET), with a margin. Pre-movement time is usually the biggest and most uncertain part.
8. **2.8 Calibration, validation and uncertainty** · 45 min
   A model is only as trustworthy as the data it was tested against. Calibrate parameters on one data set, validate on another, test standard cases, and report the uncertainty.

Mind map of the go



2.1 Measuring crowds: counts, trajectories and density

45 minutes · Data

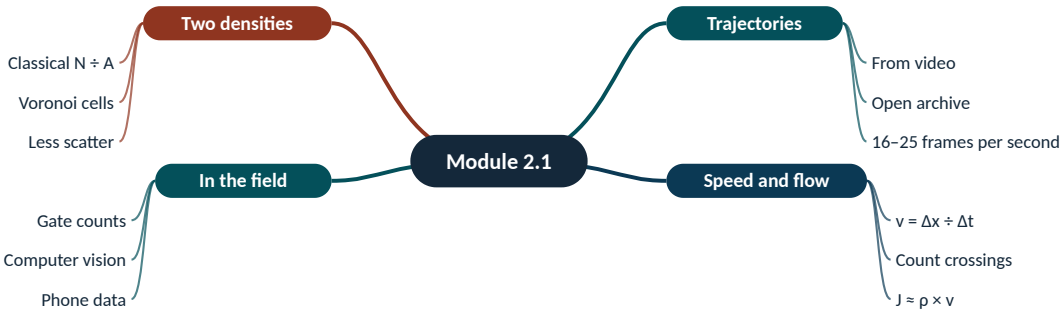
Every model starts from measurements. Learn to get density, speed and flow from counts and trajectories, and why the Voronoi method gives smoother, fairer densities than counting heads in a box.

By the end you can

- compute density, speed and flow from trajectory data
- explain classical and Voronoi density and when each is better
- check that J = ρ × v holds in your own measurements
- name the main open datasets and tools for crowd measurement

Overview

Before you can model a crowd, you have to measure it. The richest data are trajectories: the position of every person, many times a second, extracted from video. Researchers at Forschungszentrum Jülich and partners have run hundreds of laboratory experiments and published the trajectories openly. From trajectories you can compute the three basic quantities. Density can be measured the classical way, by counting heads in a measurement area and dividing by its area, but that number jumps every time someone steps in or out. The Voronoi method gives each person the patch of floor closer to them than to anyone else; density inside the area is then the average of one over each patch's size, weighted by how much of the patch lies inside. It is smoother and responds to small changes. Speed comes from the change in position over time, and flow from counting people crossing a line. In this module you will compute all three from synthetic trajectories, and check that flow equals density times speed.



Trajectories

The richest crowd data are **trajectories**: (x, y) positions of each person at each time. They come from overhead video processed by tracking software such as PeTrack, or from experiments where participants wear coloured caps. The **Jülich pedestrian dynamics data archive** publishes hundreds of such experiments.

Classical and Voronoi density

Classical density counts people in a measurement area A: ρ = N ÷ A. It jumps by 1 ÷ A each time someone enters or leaves. **Voronoi density** (Steffen and Seyfried 2010) gives each person i the region closer to them than to anyone else (area A_i) and averages:

$$\rho_V = (1 \div A) \times \Sigma_i \text{ (area of cell i inside the box)} \div A_i$$

It varies smoothly, works in small areas, and reduces scatter in fundamental diagrams.

TRY See it

In the explorer, move the seed slider: the classical density in the box jumps between snapshots, while the Voronoi density changes more smoothly.

Speed and flow

Speed is distance moved over a short time: v = Δx ÷ Δt, usually over 0.5 to 1 s to smooth out the sway of each step. Flow is the number of people crossing a line in a time window, divided by the window, and by the width for specific flow. In a steady crowd, the three must agree: **J ≈ ρ × v**. If they do not, check the measurement area, time window or tracking.

TRAP Edge effects

Voronoi cells at the edge of the crowd are unbounded. Cut them at a sensible distance or with walls, as PedPy does.

Beyond the laboratory

Real events rarely have overhead cameras everywhere. Practitioners combine **counts** (gates, turnstiles, ticket scans), **CCTV** with computer-vision counting (module 3.4), and **phone-based data** such as Wi-Fi probes or network data. Each has errors and privacy issues: report both.

LAB Density, speed and flow from trajectories
`labs/go2/m21_measure.py`

Synthetic one-way corridor. Computes classical density, mean speed, flow across a line and a Voronoi-style density for one frame. Replace the synthetic trajectories with data from the Jülich archive.

```
people crossing x = 12 m: 120 in 60 s -> flow 2.00 per s, 0.50 per m per s
mean classical density 0.42 per m2 (all frames), mean speed 1.14 m/s
check J = rho * v: 0.50 per m per s
frame 300: classical 1.00 per m2, Voronoi 0.96 per m2
```

Takeaways

- Trajectories give density, speed and flow; the Jülich archive and PedPy make them accessible.
- Classical density jumps; Voronoi density is smoother and works in small areas.
- J ≈ ρ × v is a built-in check on your measurements.
- Field data combine counts, CCTV and phone data, each with errors and privacy issues.

Check yourself

1. Classical density in a 16 m² area with 24 people is…
   (a) 0.67 per m² · (b) 1.5 per m² · (c) 24 per m² · (d) 384 per m²
2. A Voronoi cell of a person is…
   (a) Their body outline · (b) The region closer to them than to anyone else · (c) A 1 m² square · (d) The camera's view
3. Why measure speed over about a second?
   (a) Cameras are slow · (b) To smooth out the sideways sway of each step · (c) It is required by law · (d) To save memory
4. Your measured flow is much larger than density × speed. What should you suspect?
   (a) Nothing, this is normal · (b) A measurement error: area, time window or tracking · (c) The crowd is too happy · (d) Weidmann was wrong

Answers: 1 (b) 24 ÷ 16 = 1.5.; 2 (b) Each point of floor belongs to its nearest person.; 3 (b) Step sway adds noise at short intervals.; 4 (b) In steady flow they should agree.

Words

- **Trajectory**: The path of one person: their position at successive times.
- **Measurement area**: The region in which density and speed are measured.
- **Voronoi density**: Density from each person's Voronoi cell: smoother than counting heads.
- **Voronoi cell**: The region of floor closer to one person than to any other.
- **PedPy**: Open-source Python library for pedestrian data analysis.

Reading

- Jülich pedestrian dynamics data archive: Open trajectories from controlled experiments. https://ped.fz-juelich.de/da/
- PedPy (GitHub): Compute density, speed and flow, including Voronoi methods. https://github.com/PedestrianDynamics/PedPy
- Steffen and Seyfried (2010): Methods for measuring pedestrian density, flow, speed and direction: The Voronoi method and its benefits. https://arxiv.org/abs/0911.2165

2.2 Macroscopic models: crowds as a fluid

45 minutes · Model

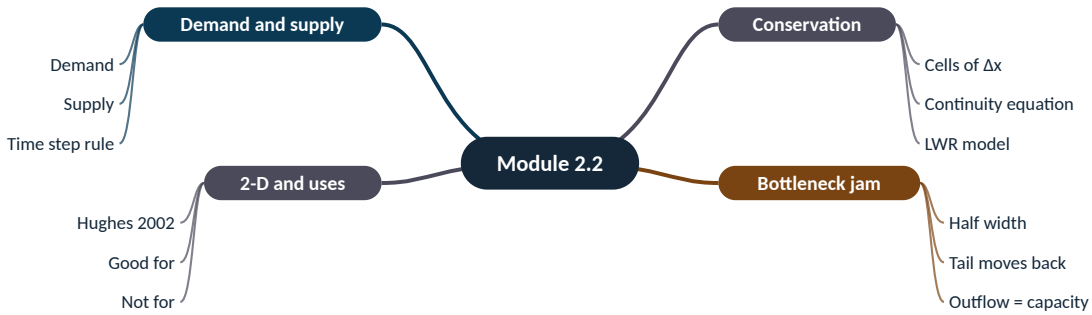
Treat the crowd as a continuous density, conserve people, and let the fundamental diagram decide the flow. A jam forms at a bottleneck and its tail travels backwards, exactly as the numbers predict.

By the end you can

- write the conservation law for crowds
- explain demand and supply between cells
- run a 1-D corridor model with a bottleneck and read the queue
- describe Hughes' model and what macroscopic models are good for

Overview

From far away, a dense crowd looks like a fluid. Macroscopic models use that view. Instead of tracking every person, they track density along a corridor or across a square. The basic rule is that people are conserved: the change in the number of people in a stretch of corridor equals those coming in minus those going out. To close the equations we need the fundamental diagram, which tells us the flow for each density. A simple, robust way to compute this is to cut the corridor into cells. Each step, the flow from one cell to the next is the smaller of two numbers: what the upstream cell can send, called demand, and what the downstream cell can receive, called supply. When the corridor narrows, the bottleneck's supply is smaller, a jam forms in front of it, and the tail of the jam moves backwards, just as you saw in Go 1. Roger Hughes extended the idea to two dimensions, with people moving down the fastest route to their goal. Macroscopic models are fast, so they suit large networks and quick what-if questions.



People are conserved

Cut a corridor into cells of length Δx. Over a time step Δt, the number of people in cell i changes by what flows in minus what flows out:

$$\rho_i(t + \Delta t) = \rho_i(t) + (\Delta t \div \Delta x) \times (f_{i-1 \to i} - f_{i \to i+1})$$

This is the discrete form of the continuity equation $\partial\rho/\partial t + \partial(\rho v)/\partial x = 0$, the basis of the LWR model (Lighthill, Whitham and Richards), first written for road traffic.

Demand, supply and the flow between cells

Which flow goes between cells? The **cell transmission** (Godunov) rule says: the smaller of what the upstream cell can **send** (demand) and what the downstream cell can **receive** (supply). Below the critical density a cell can send $J(\rho)$ and receive up to capacity; above it, it can send capacity but receive only $J(\rho)$.

$$f = \min(\, D(\rho_{up}), S(\rho_{down}) \,)$$

Keep Δt small enough that nobody crosses more than one cell per step ($v_0 \Delta t \le \Delta x$).

MODEL **Same maths as roads**

The cell transmission model (Daganzo 1994) is used for road traffic too. Crowds only need a different fundamental diagram.

A jam at a bottleneck

Make cells 40 to 45 m half as wide. Their supply is halved, so less can enter. Density in front rises past critical: a jam. Its tail moves **backwards** at a speed set by the fundamental diagram (the slope between the upstream and jammed states). With an inflow of 0.9 per m per s into a bottleneck that passes only about 0.61, the queue tail retreats by about 5 to 6 m every minute in the lab model.

TRY **Explore it**

Set the bottleneck share to 1 (no bottleneck): does any jam form at an inflow of 0.9? Then lower it step by step.

Two dimensions and where macro models shine

Hughes (2002) wrote a 2-D continuum model in which people move in the direction that minimises their travel time to the goal, taking into account the density ahead. Macroscopic models are fast and smooth, so they suit **large networks, many scenarios and real-time forecasting**. They cannot show individual choices, groups or pushing at a door: for those, use microscopic models (modules 2.3 to 2.5).

LAB **Corridor with a bottleneck (cell transmission)**

`labs/go2/m22_macro.py`

1 m cells, 0.5 s steps, Weidmann's diagram. Prints the queue tail every minute. Change the inflow or the bottleneck width w[40:45].

```
t =    60 s: queue tail at 38 m, outflow 0.57 per s, max density 4.02
...
t =   300 s: queue tail at 17 m, outflow 0.61 per s, max density 4.03
```

Takeaways

- Macroscopic models treat the crowd as a density field and conserve people.
- The flow between cells is the minimum of upstream demand and downstream supply.

- At a bottleneck a jam forms and its tail travels backwards.
- Macro models are fast and suit large networks; they cannot show individual behaviour.

Check yourself

1. In the cell transmission rule, the flow between two cells is…
   (a) The sum of demand and supply · (b) The minimum of upstream demand and downstream supply · (c) Always capacity · (d) The average density
2. A queue tail at a bottleneck moves…
   (a) Forwards · (b) Backwards · (c) Not at all · (d) Sideways
3. Why must $v_0 \Delta t \le \Delta x$?
   (a) To save time · (b) So that nobody skips a cell in one step (numerical stability) · (c) Because people walk slowly · (d) To make cells square
4. Macroscopic models are best for…
   (a) Pushing at a single door · (b) Large networks and many quick scenarios · (c) Individual route choice · (d) Group behaviour

Answers: 1 (b) f = min(D, S).; 2 (b) The jam grows upstream.; 3 (b) The CFL condition.; 4 (b) They are fast and smooth.

Words

- **Macroscopic model**: A model that describes a crowd by continuous density and flow fields.
- **Continuity equation**: $\partial\rho/\partial t + \partial(\rho v)/\partial x = 0$: conservation of people.
- **Demand (sending function)**: The flow a cell can send downstream.
- **Supply (receiving function)**: The flow a cell can accept from upstream.
- **Cell transmission model**: A discrete macroscopic model using min(demand, supply) between cells.
- **Hughes model**: A 2-D continuum crowd model where people follow the fastest route.

Reading

- Hughes (2002): A continuum theory for the flow of pedestrians: The classic 2-D macroscopic crowd model. https://doi.org/10.1016/S0191-2615(01)00015-7
- Daganzo (1994): The cell transmission model: The demand and supply rule used in the lab. https://doi.org/10.1016/0191-2615(94)90002-7
- Seyfried et al. (2009): Pedestrian flow through bottlenecks: Data to test the bottleneck behaviour. https://arxiv.org/abs/physics/0702004

2.3 Cellular automata and floor fields

45 minutes · Model

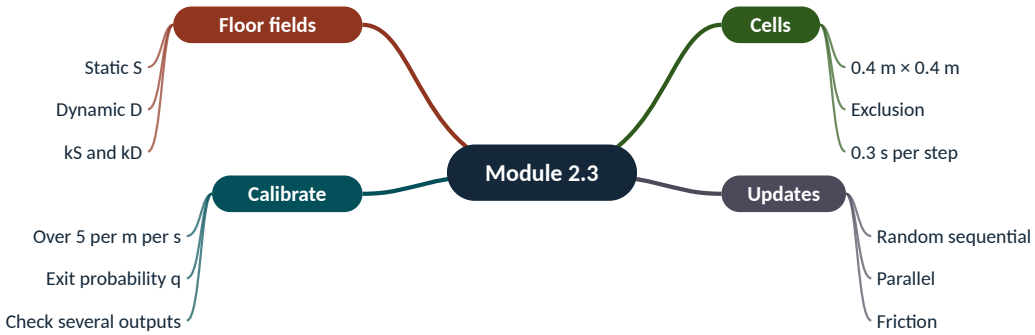
Divide the floor into 40 cm cells, one person per cell. Each step, people hop to a neighbouring cell, guided by a static field (distance to the exit) and a dynamic field (others' trails).

By the end you can

- describe the floor-field cellular automaton
- explain the roles of the static and dynamic floor fields and their weights kS and kD
- run a room evacuation and relate time steps to seconds
- spot and fix an unrealistic exit flow by calibration

Overview

A cellular automaton turns the floor into a chessboard of small cells, typically forty centimetres across, the space one person occupies in a dense crowd. Each cell is empty or holds one person. Every time step, about a third of a second, each person may hop to a free neighbouring cell. Which cell? That depends on floor fields. The static floor field is like a slope towards the exit: cells nearer the exit are more attractive. The dynamic floor field is like footprints in snow: people leave a trail that fades over time, and others are drawn to follow it, which reproduces lanes and herding. Two numbers, kS and kD, set how strongly people respond to each field. With strong kS everyone heads straight for the exit; with weak kS people wander. The model is fast and easy to code, which made it popular, as in the floor-field model of Burstedde and colleagues in two thousand and one. But it needs care: in its simplest form it lets people leave through a door much faster than real crowds do. Calibration fixes that.



A floor of cells

A cellular automaton (CA) is a grid of cells with simple local rules. For crowds, cells are usually **0.4 m × 0.4 m**, about the space of one person in a dense crowd, so full occupancy is 6.25 per m². Each cell is empty or holds one person, which automatically prevents overlaps. Time moves in steps of about **0.3 s**, giving a free speed of about 1.3 m/s.

Static and dynamic floor fields

Burstedde et al. (2001) guided people with two fields. The **static field S** measures distance to the exit and does not change. The **dynamic field D** is a virtual trail: each move adds to it, and it decays and diffuses over time. A person chooses among free neighbour cells with probability

$$p \propto \exp(-k_S \times S) \times \exp(k_D \times D)$$

Large k_S: people know the way and head straight for it. Large k_D: people follow others (herding), useful for smoke or unfamiliar buildings.

TRY Explore it

Set kS = 0: people wander randomly and take far longer. Then raise kD and watch trails form behind early leavers.

Update rules and time

In **random sequential update**, people move one at a time in a fresh random order each step; this is simple and avoids conflicts. In **parallel update** everyone moves at once, and two people may target the same cell: a conflict rule decides who wins, and a friction parameter can make both stay (Kirchner, Nishinari and Schadschneider 2003). Parallel update with friction reproduces clogging at exits.

Calibrate before you trust it

Run the plain model and count the exit flow. With an exit two cells wide (0.8 m) the lab model empties 150 people in about 37 s: over **5 people per metre per second**, far above the 1.3 to 1.9 measured in experiments. One crude fix is an exit probability: a person on an exit cell leaves with probability q per step. With q = 0.2 the flow falls to about 1.35 per m per s. Real studies calibrate several outputs at once (module 2.8).

TRAP Nice pictures, wrong numbers

A CA can look convincing while getting capacity badly wrong. Always compare flows with data before using results.

LAB Floor-field CA evacuation

`labs/go2/m23_ca.py`

Evacuates 150 people for four values of kS. Try kD = 2, or exit_w = 4. The function also takes q_exit, the probability of leaving from an exit cell.

```
kS = 0.5: 279 steps = 84 s
kS = 1: 162 steps = 49 s
kS = 2: 122 steps = 37 s
kS = 4: 121 steps = 36 s
```

LAB Calibrating the exit flow

`labs/go2/m25_compare.py`

Compares a hand calculation with the CA, and shows how an exit probability brings the CA's exit flow into the measured range. Download it with m23_ca.py and run both together.

```
cellular automaton, exit probability 0.2: 462 steps = 139 s, exit flow 1.35 per m per s
```

Takeaways

- A floor-field CA moves people between 0.4 m cells in steps of about 0.3 s.
- The static field points to the exit; the dynamic field is a fading trail; kS and kD weight them.
- Update rules and friction decide conflicts and clogging.
- The plain CA's exit flow is far too high: calibrate before use.

Check yourself

1. A 0.4 m cell, fully occupied, corresponds to a density of…
   (a) 2.5 per m² · (b) 6.25 per m² · (c) 4 per m² · (d) 16 per m²
2. A high kD makes people…
   (a) Ignore others · (b) Follow others' trails · (c) Stop moving · (d) Walk faster
3. With kS = 0, people…
   (a) Head straight to the exit · (b) Wander randomly · (c) Freeze · (d) Push harder
4. The plain lab CA gives an exit flow of about…
   (a) 0.5 per m per s · (b) 1.5 per m per s · (c) over 5 per m per s · (d) exactly 1.3

Answers: 1 (b) 1 ÷ 0.16 m² = 6.25.; 2 (b) kD weights the dynamic field.; 3 (b) No preference for the exit.; 4 (c) Much too high without calibration.

Words

- **Cellular automaton**: A grid model where each cell's state changes by local rules each step.
- **Static floor field**: A fixed field, such as distance to the exit, that attracts people.
- **Dynamic floor field**: A virtual trail left by moving people that decays and diffuses.
- **Random sequential update**: Moving agents one at a time in a random order each step.
- **Friction parameter**: In parallel-update CAs, the chance that conflicting people both stay put.

Reading

- Burstedde et al. (2001): Simulation of pedestrian dynamics using a 2-D cellular automaton: The floor-field model. https://arxiv.org/abs/cond-mat/0102397
- Kirchner, Nishinari and Schadschneider (2003): Friction effects and clogging: Friction in a CA reproduces clogging at exits. https://arxiv.org/abs/cond-mat/0209383
- Seyfried et al. (2009): Bottleneck experiments: Exit flows to calibrate against. https://arxiv.org/abs/physics/0702004

2.4 The social force model

50 minutes · Model

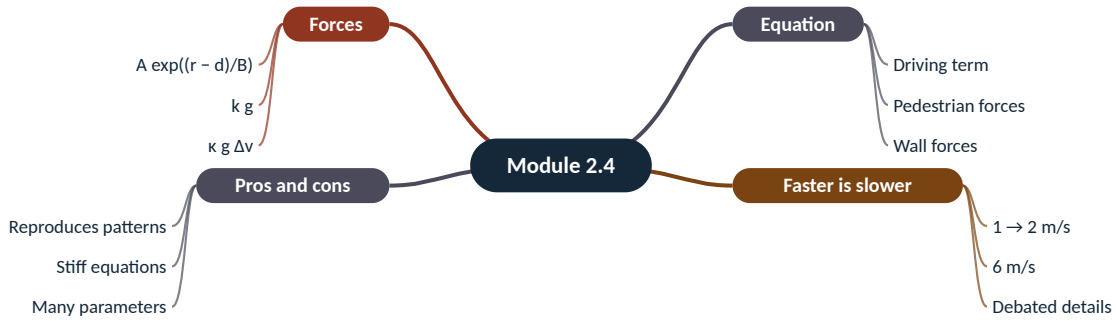
Each person is pulled towards a goal and pushed by neighbours and walls, like particles on springs. The model reproduces lanes, clogging at doors and 'faster is slower', but needs careful parameters.

By the end you can

- write the social force equation and name each term
- explain the roles of desired speed, relaxation time and repulsion
- reproduce clogging and 'faster is slower' in a simulation
- list the model's strengths and weaknesses

Overview

In nineteen ninety-five Dirk Helbing and Péter Molnár described pedestrians as if they were particles moved by forces. Not physical forces, at first, but social ones: each person feels a pull towards their goal at their desired speed, and a push away from other people and walls that grows quickly as they get closer. The pull works like a spring: if you are slower than you want to be, you accelerate, over a relaxation time of about half a second. In two thousand, Helbing, Farkas and Vicsek added physical forces for when bodies actually touch: a body force that resists squeezing, and sliding friction. With these, the model shows something surprising. When people try to leave a room very fast, they press into the door, arches of bodies form and block it, and the room empties more slowly. Faster is slower. The social force model reproduces lanes, oscillations at doors and clogging. Its weaknesses are many parameters, sensitivity to time steps, and people who can overlap or wobble if parameters are poorly chosen.



The equation

For person i with mass m, velocity v and desired velocity $v_0 e$:

$$m\, dv_i/dt = m\,(v_0 e_i - v_i) \div \tau + \Sigma_j\, f_{ij} + \Sigma_W\, f_{iW}$$

The first term pulls the person to their desired velocity over a relaxation time τ. The others are interaction forces with other people j and walls W (Helbing and Molnár 1995).

Repulsion and contact

Helbing, Farkas and Vicsek (2000) used

$$f_{ij} = [A \exp((r_{ij} - d_{ij}) \div B) + k\, g(r_{ij} - d_{ij})]\, n_{ij} + \kappa\, g(r_{ij} - d_{ij})\, \Delta v^{t}_{ji}\, t_{ij}$$

where r_{ij} is the sum of radii, d_{ij} the distance, n and t the normal and tangent, and g(x) = x when bodies overlap, else 0. The first part is social repulsion; the k and κ terms act only on contact.

Faster is slower

In the lab with 100 people and a 0.8 m door, raising the desired speed from 1 to 2 m/s halves the evacuation time (85 s to 43 s), 4 m/s helps a little (40 s), but 6 m/s is **slower** (52 s): people press into the door, form arches and block each other. Helbing, Farkas and Vicsek (2000) called this **faster-is-slower**. Experiments with pushing crowds and animals show related effects, though the details remain debated.

TRY Explore it

Run the explorer with v_0 = 1.0 m/s, then 3 m/s. Watch the door: do arches form and break?

Strengths and weaknesses

Strengths: continuous space, intuitive forces, reproduces lanes, oscillations and clogging; widely implemented. **Weaknesses:** many parameters, stiff equations needing small time steps (about 0.005 s with contact forces), people who can overlap or oscillate, and speeds that do not drop to zero at high density without extra care. Many variants exist; always state which you use and how you calibrated it.

TRAP Time step

With contact forces, a time step of 0.05 s can let people pass through each other. The lab uses 0.005 s.

LAB Social force on one pair

Repulsion between two people at different distances. Social repulsion grows exponentially; the body force switches on only when they overlap.

```
distance 1.00 m: social       13 N, body       0 N
...
distance 0.55 m: social     3736 N, body    6000 N
```

LAB Room evacuation with the social force model

`labs/go2/m24_sfm.py`

NumPy simulation of 100 people and a 0.8 m door for four desired speeds. Takes a few minutes; download and run it on your machine.

```
v0 = 1.0 m/s: 100 out, last after 85.47 s
v0 = 2.0 m/s: 100 out, last after 43.155 s
v0 = 4.0 m/s: 100 out, last after 40.47 s
v0 = 6.0 m/s: 100 out, last after 52.21 s
```

Takeaways

- The social force model: driving force + repulsion from people and walls + contact forces.
- Typical values: $\tau \approx 0.5$ s, $A \approx 2{,}000$ N, $B \approx 0.08$ m (Helbing et al. 2000).
- It reproduces lanes, clogging and faster-is-slower.
- It has many parameters and stiff equations: use small time steps and calibrate.

Check yourself

1. The driving force term is…
   (a) $m(v_0 e - v)/\tau$ · (b) $A \exp(d/B)$ · (c) $k\,g$ · (d) $\rho \times v$
2. What does τ (about 0.5 s) represent?
   (a) Body radius · (b) How quickly people adapt their velocity · (c) Range of repulsion · (d) Door width
3. 'Faster is slower' means…
   (a) Slow walkers are faster · (b) Pushing to leave faster can clog the exit and slow evacuation · (c) Speed limits help · (d) Fast music slows crowds
4. Why use a small time step with contact forces?
   (a) To save computing time · (b) The equations are stiff; large steps let people pass through each other · (c) For nicer pictures · (d) Because people walk slowly

Answers: 1 (a) It relaxes velocity towards the desired velocity.; 2 (b) The relaxation time.; 3 (b) Arches form at the door.; 4 (b) Stiff contact forces need small steps.

Words

- **Social force model**: A microscopic model where people move under a driving force and interaction forces (Helbing and Molnár 1995).
- **Desired speed**: The speed a person wants to walk at (v_0).
- **Relaxation time**: The time over which a person adapts their velocity (τ).
- **Faster-is-slower**: The effect that higher desired speeds can slow an evacuation through clogging.
- **Arching**: A blocking arch of people that forms in front of a narrow exit.

Reading

- Helbing and Molnár (1995): Social force model for pedestrian dynamics: The original model. https://doi.org/10.1103/PhysRevE.51.4282
- Helbing, Farkas and Vicsek (2000): Simulating dynamical features of escape panic: Contact forces and faster-is-slower. https://doi.org/10.1038/35035023

- JuPedSim documentation: Open-source simulator with force-based and velocity-based models. https://www.jupedsim.org/

2.5 The crowd model family: choosing the right tool

40 minutes · Compare

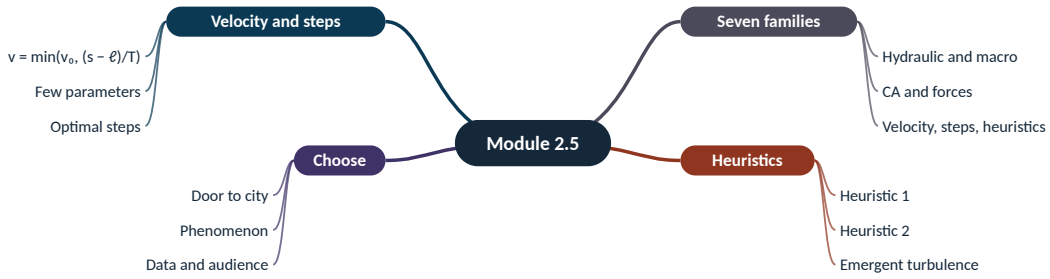
Hydraulic calculations, macroscopic, cellular, force-based, velocity-based, step-based and heuristic models each answer different questions. Choose by the question, the scale and the data you have.

By the end you can

- place each model family on a scale from hand calculation to cognitive model
- match a model family to a research or design question
- explain velocity-based and optimal-steps models
- know which open simulators implement which models

Overview

You now know three ways to model a crowd: as a fluid, as people hopping on a grid, and as particles pushed by forces. There are more. Velocity-based models choose a speed and direction directly, for example the collision-free speed model, where each person walks as fast as possible without colliding within a short time horizon. The optimal steps model, developed by Seitz and Köster, moves people in discrete steps, choosing the best point on a circle around them. Cognitive heuristic models, from Moussaïd, Helbing and Theraulaz, assume people look ahead and head where they can walk furthest towards their goal without a collision. At the other end of the scale, the hydraulic method is a hand calculation with flow rates and walking speeds, still used in building codes. No model is best for everything. The question decides: sizing an exit, forecasting a city-wide event, studying pushing at a door, or exploring how individual decisions shape a crowd. Open simulators such as JuPedSim and Vadere implement several of these models, so you can compare them on the same scenario.



Seven families

From simplest to most detailed: **hydraulic** calculations; **macroscopic** (continuum and cell-transmission) models; **cellular automata**; **force-based** models; **velocity-based** models; **optimal steps**; **cognitive heuristics**. The model atlas has an animation, a simulator and a mind map for each.

Velocity-based and step-based models

The **collision-free speed model** (Tordeux, Chraibi and Seyfried 2016) sets each person's speed to the largest value that avoids a collision within a time gap T:

$$v = \min(\, v_0, \max(0, (s - \ell) \div T)\,)$$

where s is the distance to the person ahead and ℓ the body length. It reproduces the fundamental diagram with few parameters. The **optimal steps model** (Seitz and Köster 2012) moves people in steps to the best point on a circle, scoring distance to the goal and personal space.

MODEL In the atlas

The collision-free speed model runs on a ring in the model atlas: add people and watch the fundamental diagram appear.

Cognitive heuristics

Moussaïd, Helbing and Theraulaz (2011) proposed two rules: **(1)** walk in the direction that gets you closest to your goal before you would collide with someone, scanning your field of vision; **(2)** walk at a speed that keeps a time gap τ to the first obstacle. Adding body contact forces, the same rules reproduce lanes, stop-and-go waves and turbulence at high density.

Choosing by the question

Ask: what scale (a door, a building, a city)? Which phenomenon (capacity, queues, pushing, behaviour)? Which data can calibrate it? How will decision makers understand it? For a code check, a hydraulic calculation may be right; for a city festival network, a macroscopic or CA model; for pushing at a gate, a force-based model; for how information changes behaviour, a cognitive or agent-based model with route choice.

TRY Use the explorer

Pick each question in the explorer and see how the ranking changes. Do you agree with it?

LAB Collision-free speed model on a ring

Steady state of the collision-free speed model in single file. Flow rises, peaks and falls with density: a fundamental diagram from one rule.

```
T 0.5: 15 people, density 0.58 per m, speed 1.30 m/s, flow 0.75 per s
...
T 1.0: 60 people, density 2.31 per m, speed 0.13 m/s, flow 0.31 per s
```

Takeaways

- Seven model families run from hand calculations to cognitive heuristics.
- Velocity-based models choose a collision-free speed directly; optimal steps moves people in discrete steps.
- Cognitive heuristics model what people see and decide.
- Choose by scale, phenomenon, available data and audience; open simulators let you compare.

Check yourself

1. In the collision-free speed model, speed is set by…
   (a) Social forces · (b) The gap ahead and a time gap T · (c) A floor field · (d) Random choice
2. Which family best suits pushing and clogging at a single gate?
   (a) Hydraulic · (b) Macroscopic · (c) Force-based · (d) Network flow
3. The optimal steps model moves people…
   (a) Continuously by forces · (b) In discrete steps to the best point on a circle · (c) Between grid cells only · (d) By density gradients
4. Which is a strength of hydraulic calculations?
   (a) Shows individual behaviour · (b) Transparent and fast · (c) Reproduces turbulence · (d) Needs no flow rates

Answers: 1 (b) $v = \min(v_0, (s - \ell)/T)$.; 2 (c) Contact forces matter.; 3 (b) Seitz and Köster 2012.; 4 (b) Simple and checkable by hand.

Words

- **Hydraulic method**: Egress calculation using flow rates and speeds, without individual behaviour.
- **Velocity-based model**: A model in which agents choose their velocity directly, usually to avoid collisions.
- **Collision-free speed model**: Speed = $\min(v_0, (\text{gap} - \text{body length})/T)$ (Tordeux, Chraibi and Seyfried 2016).
- **Optimal steps model**: Agents move in discrete steps to the best point on a circle (Seitz and Köster 2012).
- **Cognitive heuristics**: Simple perception-based rules for direction and speed (Moussaïd et al. 2011).

Reading

- Seitz and Köster (2012): Natural discretization of pedestrian movement in continuous space: The optimal steps model. https://doi.org/10.1103/PhysRevE.86.046108
- Moussaïd, Helbing and Theraulaz (2011): Cognitive heuristics and crowd disasters. https://doi.org/10.1073/pnas.1016507108
- Vadere crowd simulation: Open-source framework with optimal steps, social force and more. https://www.vadere.org/
- JuPedSim: Open-source simulator including the collision-free speed model. https://www.jupedsim.org/

2.6 Route and exit choice

40 minutes · Behaviour

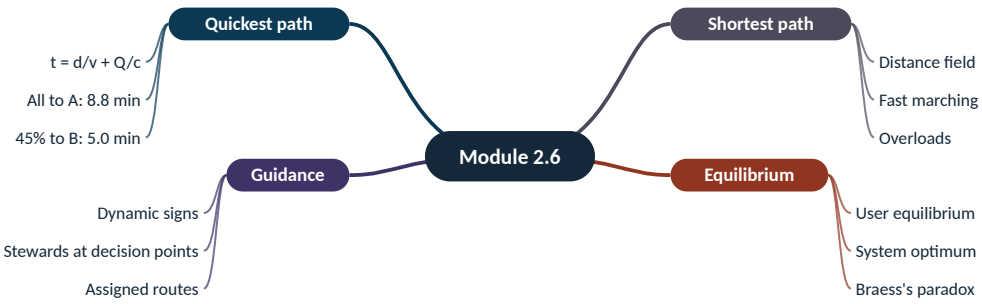
Where people go matters as much as how they walk. Shortest-path, quickest-path and equilibrium rules give different crowd patterns; good guidance moves the crowd towards the best split.

By the end you can

- compare shortest-path and quickest-path route choice
- find the best split of people between exits
- explain the difference between a user optimum and a system optimum
- describe how information and guidance shift choices

Overview

Imagine two exits: one near, one far. If everybody takes the nearest, it queues while the far one stands empty. If some take the far exit, they walk longer but wait less, and everyone gets out sooner. The best split equalises the time at which each exit finishes. Models capture this with different rules. The simplest is the shortest path: walk the fewest metres. A better rule is the quickest path: choose the route with the least expected time, walking plus waiting. If everyone chooses selfishly in this way, the crowd reaches a user equilibrium, where no one could do better by switching. The system optimum, best for everyone together, can be different, and may need guidance to reach. In practice, people also follow habit, signs and others, as we saw in Go 1. Simulators compute route choice with distance fields, for example the fast marching method, and update them as congestion grows. In this module you will find the best split between two exits and see how a simple expected-time rule can find it on its own.



Shortest path and distance fields

The simplest rule: everyone takes the shortest path. In simulators this is a **distance field**, the distance from every point to the exit, computed for example with the fast marching method, which handles obstacles. People walk down its gradient. With several exits, each person heads for the nearest, which can overload one exit badly.

Quickest path and the best split

Add waiting. A person's expected time to exit k is walking time plus the queue ahead divided by capacity:

$$t_k = d_k \div v + Q_k \div c_k$$

Example: 2,000 people, exit A at 30 m, exit B at 90 m, each passing 4 per s, speed 1.2 m/s. All to A: about 8.8 minutes. Best split: about 45% to B, everyone out in about 5.0 minutes. A simple rule where each person picks the exit with the smaller expected time finds the same split.

TRY Explore it

In the explorer, find the share that minimises the time. Is it where both exits finish at the same moment?

User equilibrium and system optimum

When everyone minimises their own time, the crowd reaches a **user equilibrium**: all used routes take the same time and no one gains by switching. The **system optimum** minimises a total, such as the time until the last person leaves or the sum of everyone's delay. In networks the two can differ (as in Braess's paradox, where adding a route can make everyone slower). Guidance aims to close the gap.

RESEARCH Research angle

How far real crowds are from the system optimum, and how much guidance can close the gap, is an active research question.

Guidance in practice

Real people do not compute expected times. They follow habit, signs, stewards and each other. Guidance that works is **early**, **visible** and **trusted**: dynamic signs showing open exits, stewards redirecting at decision points, and apps or announcements with clear instructions. At mass gatherings, routes are often **assigned** rather than chosen, by schedule and zone (module 3.3).

LAB Best split and a quickest-path rule

`labs/go2/m26_route.py`

Finds the best split by trying all shares, then lets each person choose the exit with the smaller expected time.

```
all to the nearest exit: 8.8 min
best split: 45% to the far exit, everyone out after 5.0 min
people choosing by expected time: 45% to the far exit -> 5.0 min
```

Takeaways

- Shortest-path choice can overload the nearest exit.
- Quickest-path choice adds expected waiting: t = d/v + Q/c.
- User equilibrium (selfish) and system optimum (best for all) can differ.
- Guidance must be early, visible and trusted; mass gatherings often assign routes.

Check yourself

1. In the example, sending about 45% to the far exit…
   (a) Makes everyone slower · (b) Cuts the time from about 8.8 to 5.0 minutes · (c) Has no effect · (d) Doubles the queue
2. At a user equilibrium…
   (a) Everyone uses the shortest path · (b) No one can reduce their own time by switching · (c) Total time is always minimal · (d) Nobody moves
3. A distance field gives…
   (a) The quickest path · (b) The shortest path · (c) The density · (d) The fundamental diagram
4. Braess's paradox shows that…
   (a) Adding a route can make everyone slower · (b) More exits always help · (c) Signs never work · (d) Distance does not matter

Answers: 1 (b) Walking further saves waiting.; 2 (b) Used routes take equal time.; 3 (b) Distance only, no waiting.; 4 (a) Selfish choices on a new link can raise everyone's time.

Words

- **Distance field**: The distance from every point to a target, used to steer simulated people.
- **Fast marching method**: An algorithm to compute distance (or travel-time) fields around obstacles.
- **Quickest path**: The route with the least expected travel time including waiting.
- **User equilibrium**: A state where no one can reduce their own travel time by changing route.
- **System optimum**: The routing that minimises a total measure for everyone.

Reading

- Hughes (2002): continuum crowd flow with fastest routes: Route choice inside a macroscopic model. https://doi.org/10.1016/S0191-2615(01)00015-7
- Haghani and Sarvi (2018): Crowd behaviour and motion, empirical methods: Review of experiments including exit choice. https://doi.org/10.1016/j.trb.2017.06.017
- Vadere: route choice and floor fields: Open simulator with distance fields and dynamic routing. https://www.vadere.org/

2.7 Evacuation modelling: RSET and ASET

45 minutes · Safety

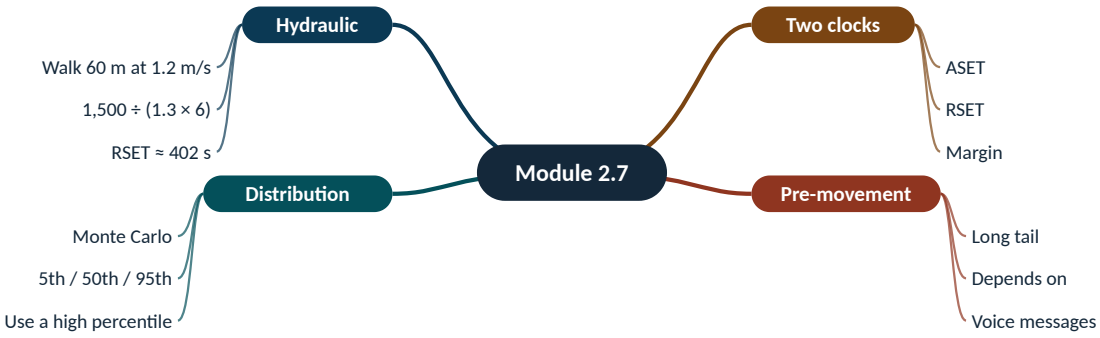
An evacuation is safe when the time people need (RSET) is less than the time available (ASET), with a margin. Pre-movement time is usually the biggest and most uncertain part.

By the end you can

- break RSET into detection, warning, pre-movement and movement
- estimate movement time with the hydraulic method
- explain why pre-movement time dominates and is uncertain
- treat RSET as a distribution, not a single number

Overview

Fire engineers ask a simple question: will everyone get out before conditions become dangerous? The time available, called ASET, comes from fire and smoke modelling. The time required, called RSET, is the sum of four parts: the time to detect the fire, the time to warn people, the pre-movement time while people notice, decide and gather their things or their families, and the movement time to walk to and through the exits. A design is acceptable when RSET is clearly shorter than ASET. Movement time can be estimated by hand with the hydraulic method: the walking time, or the time for the crowd to flow through the exits, whichever is longer. Simulators add detail. But the largest and most uncertain part is usually pre-movement. People finish their conversation, look for friends, or wait for confirmation. Experiments and real evacuations show long, skewed distributions of pre-movement times. That is why careful studies treat RSET not as one number but as a distribution, and look at its high percentiles.



Two clocks

Performance-based fire engineering compares two times. **ASET**, the available safe egress time, runs until smoke, heat or toxic gases make conditions untenable; it comes from fire modelling. **RSET**, the required safe egress time, is how long people need:

$$\mathrm{RSET} = t_{\mathrm{detection}} + t_{\mathrm{warning}} + t_{\mathrm{pre\text{-}movement}} + t_{\mathrm{movement}}$$

The design is acceptable if RSET < ASET with an adequate safety margin (see the SFPE Handbook and PD 7974-6).

Movement time by hand

The **hydraulic method** takes the longer of walking time and flow time:

$$t_{\mathrm{movement}} \approx \max\left(\ \mathrm{distance} \div \mathrm{speed},\ \mathrm{people} \div (\mathrm{specific\ flow} \times \mathrm{exit\ width})\ \right)$$

Example: 1,500 people, 6 m of exits at 1.3 per m per s, 60 m at 1.2 m/s: walking 50 s, flow 1,500 ÷ 7.8 ≈ 192 s, so movement ≈ 192 s. With detection 60 s, warning 30 s and pre-movement 120 s, RSET ≈ 402 s, about 6.7 minutes.

TRY Explore it

In the explorer, double the pre-movement time. Then double the exit width instead. Which changes RSET more?

Pre-movement time

People rarely move the moment an alarm sounds. They look for information, finish tasks, find their group. Measured pre-movement times are often **long-tailed**: most people start within a minute or two, some much later. Pre-movement depends on alertness (asleep or awake), familiarity, staff behaviour and the alarm: **voice messages** that say what to do shorten it compared with bells.

TRAP Zero pre-movement

Simulations that start everyone moving at time zero can underestimate RSET badly. Always add a realistic pre-movement distribution.

RSET as a distribution

Uncertain inputs make RSET uncertain. Sample them many times (Monte Carlo): in the lab, a lognormal pre-movement around 120 s and uncertain flow and speed give RSET percentiles of about 323 s (5th), 409 s (50th) and 569 s (95th). Compare a high percentile, not the average, with ASET, and report how sensitive the result is to each input.

LAB RSET and its uncertainty

`labs/go2/m27_rset.py`

A deterministic hydraulic RSET, then 5,000 Monte Carlo samples of uncertain pre-movement, flow and speed.

```
deterministic RSET: 402 s
RSET 5th / 50th / 95th percentile: 323 / 409 / 569 s; ASET 900 s
share of runs with RSET > ASET: 0.0%
```

Takeaways

- RSET = detection + warning + pre-movement + movement; it must be below ASET with a margin.
- The hydraulic method takes the longer of walking time and flow time.
- Pre-movement time is often the largest and most uncertain part.

- Treat RSET as a distribution and compare high percentiles with ASET.

Check yourself

1. RSET stands for…
   (a) Required safe egress time · (b) Rapid safe exit test · (c) Real smoke escape time · (d) Route safety estimate
2. 1,500 people, 6 m of exits at 1.3 per m per s. Flow time?
   (a) About 50 s · (b) About 192 s · (c) About 1,170 s · (d) About 7.8 s
3. Which part of RSET is usually most uncertain?
   (a) Detection · (b) Warning · (c) Pre-movement · (d) Walking speed
4. Why compare a high percentile of RSET with ASET?
   (a) Averages hide the slow cases that matter for safety · (b) It is easier · (c) ASET is a percentile · (d) To reduce RSET

Answers: 1 (a) The time people need.; 2 (b) 1,500 ÷ 7.8 ≈ 192 s.; 3 (c) Human response varies widely.; 4 (a) Safety depends on the tail.

Words

- **RSET**: Required safe egress time: detection + warning + pre-movement + movement.
- **ASET**: Available safe egress time, until conditions become untenable.
- **Pre-movement time**: Time from warning to the start of purposeful movement.
- **Hydraulic method**: Movement time from flow rates and walking speeds.
- **Monte Carlo**: Repeated random sampling of uncertain inputs to get a distribution of results.

Reading

- SFPE Handbook of Fire Protection Engineering: Reference for egress calculations and human behaviour in fire. https://link.springer.com/referencework/10.1007/978-1-4939-2565-0
- Ronchi et al. (2013): Verification and validation of evacuation models (NIST TN 1822): How to test evacuation models. https://nvlpubs.nist.gov/nistpubs/TechnicalNotes/NIST.TN.1822.pdf
- Kuligowski: The process of human behavior in fires (NIST TN 1632): Why people delay before evacuating. https://nvlpubs.nist.gov/nistpubs/TechnicalNotes/NIST.TN.1632.pdf

2.8 Calibration, validation and uncertainty

45 minutes · Method

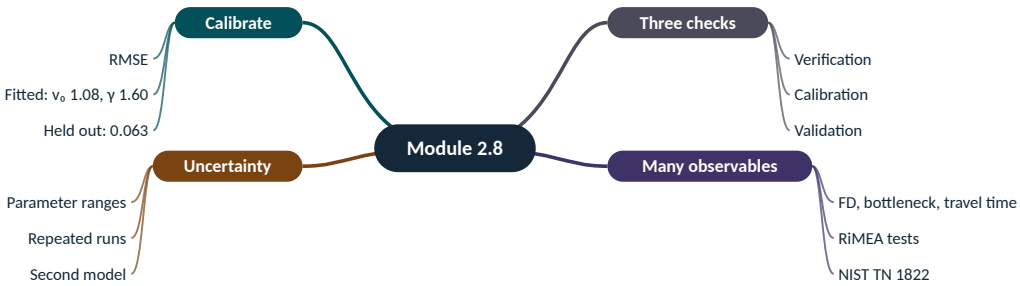
A model is only as trustworthy as the data it was tested against. Calibrate parameters on one data set, validate on another, test standard cases, and report the uncertainty.

By the end you can

- distinguish verification, calibration and validation
- calibrate a fundamental diagram by least squares
- use standard test cases such as the RiMEA tests
- report model uncertainty honestly

Overview

Every crowd model has parameters: desired speeds, repulsion strengths, floor-field weights. Calibration is choosing their values so that the model matches measurements. Validation is checking the calibrated model against different data it has not seen. Verification comes first and is simpler: does the code do what the equations say? A good calibration uses several observables at once, for example the fundamental diagram, the flow through a bottleneck and travel times, because a model can match one and miss the others. Keep part of the data aside for validation, or the model may only have memorised the data. Standard test cases, such as the RiMEA guideline tests from Germany and the NIST verification tests, check that a simulator behaves sensibly in simple situations: walking speed in a corridor, flow through a door, reaction times. Finally, report uncertainty. Run the model with plausible ranges of parameters and with more than one model if you can, and show how much the answer changes. In this module you will calibrate a fundamental diagram by hand and by computer, and validate it on held-out data.



Three different checks

Verification: is the code right? (A lone person walks at v_0; people never pass through walls.) **Calibration**: which parameter values make the model match data? **Validation**: does the calibrated model predict data it was not fitted to? Mixing them up is a common error; a perfectly calibrated but unvalidated model may simply have memorised its data.

Calibrating a fundamental diagram

Pick an error measure, often the root mean square error:

$$\mathrm{RMSE} = \sqrt{\left((1/n) \, \Sigma \, (v_{\mathrm{model}}(\rho_i) - v_i)^2 \right)}$$

and search for parameters that minimise it. In the lab, 40 'measured' points from a slower crowd give v_0 = 1.08 m/s, γ = 1.60 and ρ_{max} = 6.4 per m², with RMSE 0.055 m/s; on 20 held-out points the RMSE is 0.063. Weidmann's default curve gives 0.174: imported parameters can be badly wrong.

TRY Calibrate by hand

In the explorer, get the RMSE below 0.06 m/s. Which slider matters most at high density?

Several observables and standard tests

A model fitted only to the fundamental diagram can still get bottleneck flows or travel times wrong (recall the CA in module 2.3). Calibrate on several observables. Then run **standard tests**: the German **RiMEA** guideline tests and the NIST verification and validation tests (Ronchi et al. 2013) check simple behaviours that every simulator should get right.

Reporting uncertainty

Results depend on parameters, random seeds and model choice. Report: the parameter ranges tried, results from repeated stochastic runs (mean and spread), and, where possible, results from a second model. State what the model cannot represent (pushing, groups, information) and how that could bias the answer.

TRAP Overfitting

With enough parameters, a model can fit any one data set. Always validate on data you did not use for fitting.

LAB Least-squares calibration with validation

`labs/go2/m28_calibrate.py`

Grid search over three parameters on 40 training points, then validation on 20 held-out points.

```
fitted v0 = 1.08 m/s, gamma = 1.60, rho_max = 6.4 per m2, training RMSE 0.055 m/s
validation RMSE 0.063 m/s
Weidmann's default curve on the same data: RMSE 0.174 m/s
```

Takeaways

- Verification checks code; calibration fits parameters; validation tests on unseen data.
- Calibrate on several observables, not just one.
- Standard tests (RiMEA, NIST) check basic simulator behaviour.
- Report parameter, stochastic and model uncertainty.

Check yourself

1. Validation means…
   (a) Checking the code · (b) Fitting parameters · (c) Testing the calibrated model on data it was not fitted to · (d) Making nice plots
2. Why keep some data aside?
   (a) To save time · (b) To test for overfitting · (c) Because of privacy · (d) To make the RMSE smaller
3. Weidmann's default curve on the lab's data gave an RMSE of about…
   (a) 0.055 · (b) 0.063 · (c) 0.174 · (d) 1.34
4. RiMEA tests are…
   (a) Fire tests · (b) Standard scenarios that evacuation simulators should pass · (c) Medical tests · (d) Crowd counting methods

Answers: 1 (c) Unseen data.; 2 (b) Held-out data reveal overfitting.; 3 (c) Imported parameters can be badly wrong.; 4 (b) A German guideline.

Words

- **Verification**: Checking that a model's code correctly implements its equations.
- **Calibration**: Choosing parameter values so a model matches measured data.
- **Validation**: Testing a calibrated model against independent data.
- **RMSE**: Root mean square error, a measure of average misfit.
- **Overfitting**: Fitting noise in one data set so the model predicts new data poorly.
- **RiMEA**: German guideline with standard test cases for evacuation simulators.

Reading

- Ronchi et al. (2013): The process of verification and validation of building fire evacuation models (NIST TN 1822): Verification and validation tests. https://nvlpubs.nist.gov/nistpubs/TechnicalNotes/NIST.TN.1822.pdf
- RiMEA guideline: German guideline for microscopic evacuation analysis, with test cases. https://rimea.de/
- Jülich data archive: Data for calibration and validation. https://ped.fz-juelich.de/da/

Questions for spaced practice

From the question bank: answer from memory, then check. Revisit after 1, 3, 7 and 14 days.

2.1 Measuring crowds: counts, trajectories and density

1. **Recall:** What is a trajectory in crowd measurement?
   → The sequence of positions (x, y) of one person over time, usually extracted from video.
2. **Explain:** Why is Voronoi density smoother than classical density?
   → Because each person contributes the fraction of their cell inside the area, density changes gradually as people move, instead of jumping by 1 ÷ A when someone crosses the boundary.
3. **Calculate:** 120 people cross a line in 60 s in a 4 m wide corridor. Specific flow?
   → 120 ÷ 60 = 2 per s; ÷ 4 m = 0.5 per m per s.
4. **Explain:** How can J = ρ × v help you check data?
   → In steady flow, the measured flow should equal measured density times speed; a big mismatch points to errors in the measurement area, time window or tracking.
5. **Recall:** Name two open resources for crowd trajectories and analysis.
   → The Jülich pedestrian dynamics data archive and the PedPy Python library (PeTrack for extracting trajectories from video).
6. **Apply:** At a festival without overhead cameras, how could you estimate density?
   → Combine gate counts with area measurements, CCTV with computer-vision counting, and phone-based data, and calibrate each against manual counts.

2.2 Macroscopic models: crowds as a fluid

1. **Recall:** Write the conservation rule for people in a cell.
   → Change in people = inflow − outflow; in density form $\rho_i(t+\Delta t) = \rho_i(t) + (\Delta t/\Delta x)(f_in - f_out)$.
2. **Explain:** What are demand and supply in the cell transmission model?
   → Demand is the flow a cell can send (J(ρ) below critical density, capacity above); supply is what it can receive (capacity below critical, J(ρ) above). The flow between cells is the smaller.
3. **Calculate:** Corridor capacity 1.22 per m per s; a bottleneck half as wide. Its capacity per metre of corridor width?
   → About 0.61 per s.
4. **Explain:** Why does the tail of a jam move backwards?
   → People keep arriving at the back faster than the bottleneck releases them, so the congested region grows upstream.
5. **Recall:** What did Hughes (2002) add to macroscopic crowd models?
   → A 2-D continuum model where people move down the gradient of travel time to their goal, accounting for density ahead.
6. **Apply:** When would you choose a macroscopic model over a microscopic one?
   → For large networks, many scenarios or real-time forecasts, where speed matters more than individual detail.

2.3 Cellular automata and floor fields

1. **Recall:** What size are cells in a typical crowd CA, and why?
   → About 0.4 m × 0.4 m, roughly the space one person occupies in a dense crowd (6.25 per m² full).
2. **Explain:** What do the static and dynamic floor fields represent?
   → The static field is distance to the exit (a fixed slope towards it); the dynamic field is a fading trail of others' movements that people are drawn to follow.
3. **Calculate:** Cells of 0.4 m, steps of 0.3 s. What is the free speed?
   → 0.4 ÷ 0.3 ≈ 1.33 m/s.

4. **Explain:** Why does a plain CA overestimate exit flow?
   → With sequential update and removal at exit cells, one person can leave each exit cell every step, giving several people per metre per second, far above the 1.3–1.9 measured.
5. **Apply:** How could you make a CA's exit flow realistic?
   → Calibrate: add friction with parallel update, or an exit probability, and tune it until the exit flow matches experiments (about 1.3–1.9 per m per s).
6. **Recall:** What does kD > 0 reproduce?
   → Following and herding: people tend to step where others have just walked, which forms lanes and trails.

2.4 The social force model

1. **Recall:** Name the three kinds of term in the social force model.
   → A driving force towards the desired velocity, interaction forces with other people, and forces from walls and obstacles.
2. **Explain:** What does the relaxation time τ do?
   → It sets how quickly a person's velocity adapts to the desired velocity: the driving force is $m(v_0 e - v)/\tau$, so a small τ means a quick reaction.
3. **Explain:** How does the model produce faster-is-slower?
   → Higher desired speeds push people into the door harder; contact and friction forces form arches that block the exit, so the outflow becomes intermittent and slower.
4. **Recall:** Give typical values for A, B and τ.
   → A ≈ 2,000 N, B ≈ 0.08 m, τ ≈ 0.5 s (Helbing, Farkas and Vicsek 2000).
5. **Explain:** Why does the social force model need small time steps?
   → Contact forces are very stiff ($k \approx 10^5$), so large steps let people overlap or pass through each other; steps of about 0.005 s keep it stable.
6. **Apply:** Give two weaknesses to mention when reporting social force results.
   → Many parameters that need calibration, and numerical sensitivity (time step, overlaps, oscillations); also speeds may not fall realistically at high density without extra terms.

2.5 The crowd model family: choosing the right tool

1. **Recall:** List the seven model families in order of detail.
   → Hydraulic, macroscopic, cellular automata, force-based, velocity-based, optimal steps, cognitive heuristics.
2. **Explain:** How does the collision-free speed model choose a speed?
   → As the largest speed that avoids a collision within time T: $v = \min(v_0, \max(0, (s - \ell)/T))$, with s the gap and ℓ the body length.
3. **Recall:** What are the two heuristics of Moussaïd, Helbing and Theraulaz?
   → Choose the direction that brings you closest to your goal before a collision, and walk at a speed that keeps a time gap to the first obstacle.
4. **Apply:** Which family for a city-wide festival network with many scenarios?
   → A macroscopic or network model (or a calibrated CA) for speed, possibly with microscopic models for hot spots.
5. **Explain:** Why compare models on the same scenario?
   → Different models can give different results for the same layout; comparing them shows how much conclusions depend on model choice, a form of model uncertainty.
6. **Recall:** Which open simulator implements the optimal steps model?
   → Vadere (JuPedSim implements velocity-based and force-based models).

2.6 Route and exit choice

1. **Recall:** What is a quickest-path route choice rule?
   → Choose the route with the smallest expected time: walking time plus expected waiting time.

2. **Calculate:** Exit A: 30 m, exit B: 90 m, 4 per s each, 1.2 m/s, 2,000 people, all to A. Time?
   → 2,000 ÷ 4 = 500 s plus 25 s walking ≈ 525 s ≈ 8.8 min.
3. **Explain:** Why is the best split the one where both exits finish together?
   → If one exit finished earlier, moving someone to it from the other would shorten the total; at the optimum no move helps, so finishing times are equal.
4. **Recall:** Define user equilibrium and system optimum.
   → User equilibrium: no one can reduce their own time by switching. System optimum: the routing that minimises a total measure for all.
5. **Explain:** What does Braess's paradox teach crowd planners?
   → Adding a new route can increase everyone's travel time if people choose selfishly; more connections are not always better.
6. **Apply:** How would you guide people towards the better exit?
   → Early, visible, trusted guidance: dynamic signs showing waiting times or open exits, stewards at decision points, and clear announcements.

2.7 Evacuation modelling: RSET and ASET

1. **Recall:** List the four parts of RSET.
   → Detection, warning, pre-movement and movement time.
2. **Calculate:** Detection 60 s, warning 30 s, pre-movement 120 s, movement 192 s. RSET?
   → 60 + 30 + 120 + 192 = 402 s ≈ 6.7 min.
3. **Explain:** Why is pre-movement time often the largest uncertainty?
   → It depends on human recognition and response (alertness, familiarity, staff, the alarm message, groups), which vary widely and give long-tailed distributions.
4. **Calculate:** Movement: 60 m at 1.2 m/s, or 1,500 people through 6 m at 1.3 per m per s. Which applies?
   → Walking 50 s, flow about 192 s, so movement ≈ 192 s (the longer).
5. **Explain:** Why treat RSET as a distribution?
   → Inputs such as pre-movement, speeds and flows are uncertain; sampling them shows the range of outcomes, and safety depends on the slow tail.
6. **Apply:** How could you shorten RSET without widening exits?
   → Faster detection, clearer voice warnings with instructions, trained staff who start the evacuation, which all reduce pre-movement time.

2.8 Calibration, validation and uncertainty

1. **Recall:** Distinguish verification, calibration and validation.
   → Verification: code implements the equations. Calibration: fit parameters to data. Validation: test the calibrated model on independent data.
2. **Explain:** Why calibrate on several observables at once?
   → A model can match one quantity (such as the fundamental diagram) while getting others (bottleneck flow, travel times) wrong; several targets constrain the parameters better.
3. **Calculate:** Model speeds 1.0, 0.8, 0.5; measured 1.1, 0.7, 0.5. RMSE?
   → Errors −0.1, 0.1, 0; RMSE = √((0.01 + 0.01 + 0)/3) ≈ 0.082 m/s.
4. **Explain:** What is overfitting and how do you detect it?
   → Fitting noise in the calibration data so well that predictions on new data are poor; detect it by validating on held-out data.
5. **Recall:** Name two sources of standard tests for evacuation simulators.
   → The German RiMEA guideline and the NIST verification and validation tests (TN 1822).
6. **Apply:** What would you report about uncertainty in a simulation study?
   → Parameter ranges and sensitivity, spread across repeated stochastic runs, results from a second model if possible, and

what the model cannot represent.